

SQL Revision Notes

GATE SMASHERS LECTURE GUIDE – FINDING N-TH HIGHEST SALARY

1. Problem Overview & Placement Importance

Finding the **N-th Highest Salary** (1st, 2nd, 3rd, ... N-th) is one of the most frequently asked SQL problems in placement interviews, technical screenings, and competitive exams. The goal is to write a generic, robust query that works across all dataset variations—including duplicate salaries.

2. Sample Dataset Construction (Emp Table)

To ensure full proof testing, the dataset must include duplicate values to prevent edge-case failures during evaluation.

EMP Table Baseline

E_ID	Salary	Rank / Notes
1	10,000	Lowest
2	20,000	Duplicate Rank 4
3	20,000	Duplicate Rank 4
4	30,000	3rd Highest
5	40,000	2nd Highest
6	50,000	1st (Highest)

Salaries Summary & Ranks

Distinct Salaries ordered descending:

- **1st Highest:** 50,000
- **2nd Highest:** 40,000
- **3rd Highest:** 30,000
- **4th Highest:** 20,000

Why duplicate data matters: If duplicates (e.g., 20,000 for E_ID 2 & 3) aren't handled with `DISTINCT`, standard row counting breaks.

3. Introductory / Naive Methods (1st & 2nd Highest)

A. Finding 1st Highest Salary

Uses the standard `MAX()` aggregate function:

```
SELECT MAX(salary)
FROM Emp;
```

Output: 50,000

B. Finding 2nd Highest Salary (Nested Subquery Method)

Excludes the top salary using a nested non-correlated subquery, then computes the maximum of the remaining pool:

```
SELECT MAX(salary)
FROM Emp
WHERE salary NOT IN (
    SELECT MAX(salary)
    FROM Emp
);
```

How it works:

The inner query returns 50,000. The outer query filters out 50,000 and calculates `MAX()` on {10000, 20000, 20000, 30000, 40000}, yielding **40,000**.

Limitation of Naive Method:

This approach becomes extremely tedious and hard to write for 3rd, 4th, or N-th highest salaries (requiring `$N-1$`

4. Generic & Full-Proof Method: Correlated Subquery

To find the **N-th Highest Salary**, we count how many *distinct* salaries in the table are strictly greater than the current employee's salary (\$e1\$). When that count equals \$N-1\$, the outer row is our target.

Master N-th Highest Salary Query

```
SELECT id, salary
FROM Emp e1
WHERE (N - 1) = (
    SELECT COUNT(DISTINCT salary)
    FROM Emp e2
    WHERE e2.salary > e1.salary
);
```

5. Step-by-Step Execution Trace

Correlated subqueries evaluate **top-down** (row-by-row from \$e1\$ against the full \$e2\$ table).

Trace Example 1: Finding 4th Highest Salary (\$N = 4 \implies N-1 = 3\$)

We test rows from \$e1\$ to find where $\text{COUNT}(\text{DISTINCT } e2.\text{salary} > e1.\text{salary}) = 3$:

e1 Row (ID, Salary)	Subquery Comparison (e2.salary > e1.salary)	Distinct Salaries Greater	Count	3 = Count?	Result
Row 1: (1, 10,000)	20k, 30k, 40k, 50k are greater	{20k, 30k, 40k, 50k}	4	3 = 4 ❌	Reject
Row 2: (2, 20,000)	30k, 40k, 50k are greater	{30k, 40k, 50k}	3	3 = 3 ✅	MATCH! Output: (2, 20,000)
Row 3: (3, 20,000)	30k, 40k, 50k are greater	{30k, 40k, 50k}	3	3 = 3 ✅	MATCH! Output: (3, 20,000)
Row 4: (4, 30,000)	40k, 50k are greater	{40k, 50k}	2	3 = 2 ❌	Reject

Trace Example 2: Finding 3rd Highest Salary (\$N = 3 \implies N-1 = 2\$)

We look for rows where $\text{COUNT}(\text{DISTINCT } e2.\text{salary} > e1.\text{salary}) = 2$:

- For Row 4 (ID 4, Salary 30,000): Distinct salaries greater than 30,000 are {40,000, 50,000}.
- Count = 2. Match condition $2 = 2$ is **TRUE** ✅.
- Output: ID: 4, Salary: 30,000.

6. Key Concepts & Exam Checklist

Why Aliases (e1, e2) are Required

Since both inner and outer queries reference the Emp table, aliases prevent compiler ambiguity in condition `e2.salary > e1.salary`.

Role of DISTINCT Keyword

Without `DISTINCT`, duplicate salaries would inflate the count, leading to wrong ranks (e.g., duplicate 20k rows counting twice).

🧠 Quick Intuition Summary

- 1st Highest:** 0 salaries are greater than it (\$N-1 = 0\$).
- 2nd Highest:** 1 distinct salary is greater than it (\$N-1 = 1\$).
- N-th Highest:** Exactly \$(N-1)\$ distinct salaries are greater than it.